

CSCE 711 APPLIED CRYPTOGRAPHY · MODULE 10 · LECTURE NOTES

Key Management

The real subject, and the one with no clean answer.

MODULE OUTCOMES

- Explain the key lifecycle and each stage's failure.
- Design envelope encryption and explain why.
- Explain rotation and what it does and does not accomplish.
- Explain the architectural choice around end-to-end encryption.
- Specify a complete key management design.

1 The lifecycle

Stage	The requirement	How it actually fails
Generate	From a CSPRNG, at the full specified length.	Weak or predictable randomness (Module 02 §3), which is silent and retroactive.
Store	Somewhere only the authorised user can read it.	Committed to the repository, baked into a container image, left in a configuration file (CSCE 701 Module 07).
Distribute	Over an authenticated and confidential channel.	Emailed, pasted into chat, attached to a ticket — all of which retain it indefinitely and searchably.
Use	In memory only, for no longer than necessary.	Logged, captured in a crash dump, included in a trace or an error report — which is how keys reach third-party services.
Rotate	On a schedule, and immediately on suspected compromise.	Never, because the procedure was never tested and nobody is willing to try it under pressure.
Destroy	Verifiably, including every backup and replica.	Omitted entirely — see the note.

Destruction is the stage always omitted, and it is the one that decides whether deleting data actually deletes it. **If you retain the key, you retain the plaintext** — so a deletion policy that leaves keys in a backup has not deleted anything, which matters legally as well as technically. **§2's crypto-shredding is the mechanism that makes deletion real**, and it only works if the key destruction is genuinely complete.

A KEY IS ONLY AS PROTECTED AS THE WEAKEST PLACE IT EXISTS

Enumerate every location it has ever occupied: the key store, the running process's memory, the backup of that process's host, the developer's laptop where it was tested, the CI system that deployed it, the log line that captured it once eighteen months ago.

And the adversary needs only one of them — which is CSCE 701 Module 01's defender's asymmetry arriving as a concrete key management requirement rather than as a general principle.

So the design question is not 'is the key store secure' but 'what is the complete list of places this key has ever existed' — and that list is reliably longer than the architecture diagram shows, because the diagram records the intended flow rather than the historical one.

Which is why the best available answer is for the key never to be in your process at all. A platform key service performs the cryptographic operation and returns only the result, so the key has exactly one location, never crosses a network in plaintext, and cannot be captured from your memory or your logs — and every use is audited for free (CSCE 701 Module 09 §2).

2 Envelope encryption

ENCRYPT

```
generate a fresh data key (random, per object)
encrypt the data with it (AEAD, Module 04)
encrypt the data key with the master key
store: [encrypted data key] + [ciphertext]
```

DECRYPT

```
decrypt the data key using the master key
decrypt the data using the data key
discard the data key from memory
```

WHY THIS PATTERN IS UNIVERSAL

```
the master key encrypts only small values, so it can
  live inside a key service and never leave it
rotating the master key means re-encrypting only the
  small data keys -- not the data itself
a leaked data key exposes exactly one object
and destroying a data key destroys that object's
  plaintext permanently (crypto-shredding), which
  is how deletion becomes verifiable
```

Rotating the master key re-encrypts kilobytes rather than terabytes — which is what makes rotation operationally possible at all, and is the reason this pattern is universal in cloud key management. A system that encrypts every object directly under one master key cannot rotate that key without rewriting all of its data, which means in practice it will never rotate it, which means §3's tested procedure does not exist.

3 Rotation

ROTATION LIMITS THE DAMAGE WINDOW; IT DOES NOT UNDO A COMPROMISE

What it does: bounds how much data any single key protects, limits the window during which a leaked key remains useful, satisfies compliance requirements, and — critically — proves that the procedure actually works.

What it does not do: recover data an adversary has already decrypted, or protect anything encrypted under the old key if the adversary retained a copy of both the key and the ciphertext. Rotation is forward-looking only, and describing it otherwise is the common overclaim.

And the real value is the tested procedure. An organisation that rotates quarterly can rotate in an hour during an incident; one that has never rotated cannot — and the incident is precisely when the capability is needed and when learning it is most expensive.

So rotate on a schedule in order to be able to rotate on demand — which is CSCE 701 Module 07 §3's argument, and is genuinely the only reason scheduled rotation is worth its cost. The compliance requirement is a side effect; the capability is the point.

- **Support two keys simultaneously:** decrypt with either the old or the new key, and encrypt only with the new — which requires a key identifier stored alongside every ciphertext so you know which to try.
- **Which means the key identifier must be in the stored format from the start**, exactly as the scheme identifier had to be in Module 09 §4 — **the same design lesson, in a second place**, and adding either one retroactively is the painful migration.
- **Then re-encrypt lazily on access, or with a background pass over the data, and only remove the old key when nothing references it any longer.**
- **And verify that by counting rather than by assuming** — a query for ciphertexts still bearing the old key identifier, run to zero. **Which the key identifier makes a one-line query and which is otherwise impossible.**
- **A rotation that cannot be verified complete is a rotation you will not finish** — it will sit at 'mostly done' indefinitely, with the old key retained just in case, which forfeits the entire benefit. **The key identifier is the one design decision that makes everything else possible, and it costs two bytes.**

4 The architectural choice

END-TO-END ENCRYPTION IS AN ARCHITECTURAL DECISION WITH REAL COSTS

If the server holds the keys, it can search, index, deduplicate, scan for abuse, recover accounts, and process the data server-side — and it can be compelled by legal process or compromised into disclosure. Both halves of that are true, and the capabilities are real product features rather than conveniences.

If only the clients hold the keys, the server cannot read the data at all — and therefore cannot provide server-side search, server-side processing, abuse detection on content, or account recovery. These are not implementation gaps; they are consequences.

And the lost-key problem is the genuinely hard part. True end-to-end encryption means a user who loses their key loses their data permanently — which is unacceptable to most users — so every real system adds a recovery mechanism, and that mechanism is then the weakest point of the design and the thing an adversary or a legal process targets.

So the honest position is to state which you chose and what it forecloses — 'end-to-end encrypted, with a recovery mechanism that can be compelled' is a complete and honest claim, and 'end-to-end encrypted' alone is not. Which is Module 13's practice, arriving in the place where the marketing pressure to overclaim is strongest.

- Every key in the system, with its single purpose — and nothing serving two purposes (Module 09 §3's context labels make this nearly free).
- For each key: where it is generated, where it is stored, where it is used, and precisely which components and people can read it.
- The rotation procedure, and the date it was last actually executed — not the date it was last documented, which is the number usually reported.
- What happens when each key is exposed — the blast radius, stated per key. This is what makes designing a key hierarchy worth the effort, because it is where you discover that one key's exposure compromises everything.
- And the destruction procedure, including every backup — because 'we deleted the data' is simply false if a backup somewhere holds both the key and the ciphertext (§1's note). This specification is Project 2's deliverable, and it is the artefact that distinguishes a design from an intention.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Google — Building Secure and Reliable Systems, the key management chapters (free PDF)	§1 through §3 from operators — and free in full. The strongest practical treatment available.

Resource	What it covers for this module
AWS KMS and Google Cloud KMS design documentation (free)	§2's pattern as implemented at scale — read the envelope encryption and key hierarchy sections regardless of which platform you use.
Google Tink (free)	§2 and §3 as a library — keyset rotation and key identifiers are built in, which is the design argument made concrete.
NIST SP 800-57 — Key Management Recommendations (free)	§1's lifecycle, exhaustively — reference rather than reading, and useful for the stages people skip.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Walk the six stages** for one key in a system you operate, and name the current failure.
- 2 **Enumerate every location** one of your keys has ever existed.
- 3 **Grep your logs and traces** for anything key-shaped.
- 4 **Implement envelope encryption** with a data key per object.
- 5 **Rotate the master key** and measure how much data you re-encrypted.
- 6 **Add a key identifier** to a stored format that lacks one, and note the migration cost.
- 7 **Execute a full rotation** and verify completion by counting.
- 8 **Crypto-shred one object** by destroying its data key, and confirm the plaintext is unrecoverable.
- 9 **Write the end-to-end trade-off** for your own system, with what each choice forecloses.
- 10 **Write the complete key specification** — this is Project 2.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the six stages and the characteristic failure of each.
- 2 Why is destruction the stage that decides whether deletion is real?
- 3 Why is a key only as safe as its weakest location?
- 4 Why is a platform key service the best answer?
- 5 Describe envelope encryption and give four reasons for it.
- 6 Why does it make rotation possible?
- 7 State what rotation does and does not accomplish.

- 8 Why rotate on a schedule at all?
- 9 How do you rotate without an outage, and what must exist first?
- 10 State the end-to-end trade-off and the honest form of the claim.